EVENT

EVENT NƏDİR?

- Event — sistemdə **baş verən hər hansı hadisəni təsvir edən məlumat obyektidir.**
- Məsələn:
- “İstifadəçi qeydiyyatdan keçdi”
- “Sifariş yaradıldı”
- “Kitab alındı”

- Event **öz-özünə heç bir iş etmir.**
  Sadəcə “bu baş verdi” deyir.

EVENT NƏ ÜÇÜN İSTİFADƏ OLUNUR?

✅ **Sistemi modullar üzrə bölmək üçün**
(Hər şey bir-birinə bağlı olmasın.)
✅ **Bir hadisəyə birdən çox reaksiya vermək üçün**
Məs:
Sifariş yaradıldı → email göndər
Sifariş yaradıldı → log yaz
Sifariş yaradıldı → statistikanı yenilə

✅ **Kodda "if else hell"dən xilas olmaq üçün**
Əgər EventBus olmasa:

```
if(sifarisOluubsa) emailGonder();
if(sifarisOluubsa) logYaz();
if(sifarisOluubsa) statistika();
```

✅ **Loose Coupling yaratmaq üçün**

(Servislər bir-birini tanımasın.)

✅ **Mikroservislərdə əlaqə yaratmaq üçün**

(OrderService → EmailService → NotificationService)

PUBLISHER & SUBSCRIBER NƏDİR?

- Publisher → Event yaradan tərəf

```
eventBus.Publish(sendEvent);
```

- Subscriber → Event-i dinləyən tərəf

```
eventBus.Subscribe(new SendOrderEmailHandler());

eventBus.Subscribe(new OrderLogHandler());


eventBus.Subscribe(new UpdateOrderStatsHandler());
```

EVENTBUS NƏDİR?

- Eventləri paylayan mərkəz də demək olar.
- ✅ **Publisher event-i verir**
- ✅ **EventBus “kim dinləyir” deyə yoxlayır**
- ✅ **Hər bir subscriber-i 1-1 çağırır**

```
foreach (var handlerObj in _handlers[eventType])
{
    var handler = (IEventHandler<TEvent>)handlerObj;
    handler.Handle(@event);
}
```

- Tam olaraq “Queue-dan 1-1 oxuyub icra etmək” məntiqidir.

```
10 references
public record OrderCreatedEvent(int OrderId,int UserId,string ProductName,decimal Price);

9 references
public interface IEventHandler<TEvent>
{
    4 references
    public void Handle(TEvent @event);
}

3 references
public interface IEventBus
{
    2 references
    void Publish<TEvent>(TEvent @event);
    4 references
    void Subscribe<TEvent>(IEventHandler<TEvent> handler);
}
```

```
1 reference
public class InMemoryEventBus : IEventBus
{
    private readonly Dictionary<Type, List<object>> _handlers = new();
    2 references
    public void Publish<TEvent>(TEvent @event)
    {
        var eventType = typeof(TEvent);

        if (!_handlers.ContainsKey(eventType))
            return;

        foreach (var handlerObj in _handlers[eventType])
        {
            var handler = (IEventHandler<TEvent>)handlerObj;
            handler.Handle(@event);
        }
    }

    4 references
    void IEventBus.Subscribe<TEvent>(IEventHandler<TEvent> handler)
    {

        var eventType = typeof(TEvent);

        if (!_handlers.ContainsKey(eventType))
            _handlers[eventType] = new List<object>();

        _handlers[eventType].Add(handler);

    }
}
```

```
1 reference
public class SendOrderEmailHandler : IEventHandler<OrderCreatedEvent>
{
    2 references
    void IEventHandler<OrderCreatedEvent>.Handle(OrderCreatedEvent @event)
    {
        Console.WriteLine($"Email sent to user {@event.UserId} for order {@event.OrderId}");
    }
}

1 reference
public class OrderLogHandler : IEventHandler<OrderCreatedEvent>
{
    2 references
    void IEventHandler<OrderCreatedEvent>.Handle(OrderCreatedEvent @event)
    {
        Console.WriteLine($"Order logged: {@event.OrderId} - {@event.ProductName}");
    }
}


1 reference
public class UpdateOrderStatsHandler : IEventHandler<OrderCreatedEvent>
{
    2 references
    void IEventHandler<OrderCreatedEvent>.Handle(OrderCreatedEvent @event)
    {
        Console.WriteLine($"Order stats updated for {@event.OrderId}");
    }
}
```

```
IEventBus eventBus = new InMemoryEventBus();

eventBus.Subscribe(new SendOrderEmailHandler());

eventBus.Subscribe(new OrderLogHandler());


eventBus.Subscribe(new UpdateOrderStatsHandler());

var sendEvent = new OrderCreatedEvent(1, 1, "Product", 3);

eventBus.Publish(sendEvent);
```

QEYD

Mediator-da Send() metodu məhz bunu edir — amma EventBus-dan TAM FƏRQLİ şəkildə!
Send() Bu işi tək bir nəfər görsün və mənə cavab qaytarsın.
Yəni mediator tapşırığı götürür və tək bir handler ə yönləndirir.